

Day 19: Operator Overloading & Type-safe Registers

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Operator Overloading Basics

C++ allows you to define what built-in operators (`+`, `-`, `|=`, `[]`, etc.) mean for your custom types:

```
class Reg32 {
public:
    Reg32 &operator|=(uint32_t mask) { *reg_ |= mask; return *this; }
    Reg32 &operator&=(uint32_t mask) { *reg_ &= mask; return *this; }
    operator uint32_t() const { return *reg_; }
};
Reg32 gpio_out(&NRF_P0->OUT);
gpio_out |= BIT(13); // Looks like register access, but type-safe
```

Type-safe GPIO Pin Wrapper

Without type safety, you can accidentally pass the wrong pin number. With a wrapper class, the compiler catches mistakes:

```
class PinNumber {
public:
    explicit PinNumber(uint8_t n) : n_(n) {}
    uint8_t get() const { return n_; }
    // Cannot compare PinNumber to raw int – prevents bugs
private:
    uint8_t n_;
};
void configure_pin(PinNumber pin, bool output); // Can't accidentally pass 0x13 as a bool
```

Register Class with |= &= Operators

A register wrapper class makes hardware register access readable, type-safe, and debuggable:

```
class Reg32 {
    volatile uint32_t *reg_;
public:
    explicit Reg32(volatile uint32_t *addr) : reg_(addr) {}
    Reg32 &operator|=(uint32_t m) { *reg_ |= m; return *this; }
    Reg32 &operator&=(uint32_t m) { *reg_ &= m; return *this; }
    Reg32 &operator^=(uint32_t m) { *reg_ ^= m; return *this; }
    void set_bit(int b) { *reg_ |= (1UL << b); }
    void clr_bit(int b) { *reg_ &= ~(1UL << b); }
    bool tst_bit(int b) const { return (*reg_ >> b) & 1U; }
    operator uint32_t() const { return *reg_; }
};
```

Strong Typedef Pattern

A **strong typedef** creates a new type that is not implicitly convertible to/from its underlying type:

```
class Milliseconds {
    uint32_t v_;
public:
    explicit constexpr Milliseconds(uint32_t v) : v_(v) {}
    uint32_t value() const { return v_; }
};
// You cannot accidentally add milliseconds and ticks:
Milliseconds ms(500);
uint32_t ticks = ms; // ERROR: no implicit conversion
```

Preventing Implicit Conversions

Use `explicit` on constructors to prevent the compiler from silently creating objects:

```
class SpiClock {
    explicit SpiClock(uint32_t hz) : hz_(hz) {}
};
SpiClock clk = 1000000; // ERROR: explicit prevents this
SpiClock clk(1000000); // OK: direct construction
```

Units Library Concept

User-defined literals (`operator""_ms`) create type-safe units at compile time:

```
constexpr Milliseconds operator""_ms(unsigned long long v) { return Milliseconds(v); }  
auto period = 500_ms;    // Type: Milliseconds, value: 500  
auto debounce = 50_ms;  
auto total = period + debounce; // 550_ms  
k_msleep(total.value());
```

Practice Tasks

1. Create a `Frequency` strong typedef with `operator""_Hz` and `operator""_kHz` literals
2. Write a `GpioPort` class that wraps `NRF_P0` with `|=`, `&=`, `^=` operators for direct register access
3. Implement a `Color` class for an RGB LED with `operator+` to mix colors
4. Create `operator<` and `operator>` for your `Milliseconds` type — use it to compare timeouts

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day19  
west build -b nrf52840dk/nrf52840 .  
west flash  
# View output via RTT or UART serial (115200 baud)  
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)

- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day19/main.cpp](#)